

A Static Analysis Framework for Detecting Smart Contract Business Logic Vulnerabilities

스마트 컨트랙트 비즈니스 로직 취약점 탐지를 위한 정적 분석 프레임워크

서울여자대학교 정보보호학과 육은서

Table of Contents

01 DeFi 개요

02 관련 연구

03 제안 방법

04 평가 결과

05 결론 및 향후 연구

DeFi 개요

스마트 컨트랙트의 특이성과 비즈니스 로직

스마트 컨트랙트의 불변성으로 인해 패치가 어려움

→ 배포 전 audit이 중요

```
function withdraw(uint256 shares) external {  
    uint256 assets = convertToAssets(shares);  
  
    _burn(msg.sender, shares); // Effect  
    token.transfer(msg.sender, assets); // Interaction  
    accrueInterest();  
  
}
```

```
function withdraw(uint256 shares) external {  
    uint256 assets = convertToAssets(shares);  
  
    accrueInterest();  
    _burn(msg.sender, shares); // Effect  
    token.transfer(msg.sender, assets); // Interaction  
  
}
```

비즈니스 로직 결함은 기존 의도와 다르게 진행되어 손실을 일으키는 취약점
문법 오류가 아니기 때문에 정적 분석으로 탐지가 어려움

관련 연구

스마트 컨트랙트의 특이성과 비즈니스 로직

스마트 컨트랙트의 불변성으로 인해 패치가 어려움
→ 배포 전 audit이 중요



static analysis

AST·CFG·IR 로 변환해 취약점 패턴을 신속
히 탐지하나, 키워드·패턴 기반 규칙으로 비즈
니스 로직을 포괄적으로 검증하기 어려움



symbolic execution analysis

실행 경로를 탐색해
논리적 결함을 찾을 수 있으나,
복잡한 컨트랙트에서는 상태 공간 폭발로
시간·자원 소모가 급증



dynamic analysis (fuzzing)

정의된 속성에 따라 입력을 생성해
런타임 위반을 유도하나,
탐지 성능이 속성 정의의 완전성에 크게 의존

제안 방법

Lending protocol

해당 프로토콜에서 정의하는 핵심 함수들

DeFi에서 암호화폐 예금 및 대출 서비스를 제공하는 시스템/프로토콜



deposit()

- 자산을 입금하고 share 토큰을 받음

mint()

- 특정 수량의 share를 민팅

withdraw()

- 자산을 인출하고 share 소각

redeem()

- share를 반환하고 자산을 받음

제안 방법

ERC-4626 : vault의 표준 인터페이스

해당 함수에서 정의하는 핵심 함수들

deposit()

- MUST revert if all of assets cannot be deposited

mint()

- MUST revert if all of shares cannot be minted

withdraw()

- SHOULD check msg.sender can spend owner funds, assets needs to be converted to shares and shares should be checked for allowance.

redeem()

- MUST support a redeem flow where the shares are burned from owner directly where owner is msg.sender.



해당 금융 비즈니스 로직에서 결함 다수 발견

→ERC-4626 docs에서 요구하는 MUST, SHOULD에 대한 코드 레벨 audit 필요

제안 방법

탐지 기준 설계

Pattern match

- 하드코딩된 문자열로 필요한 함수명과 로직명 (require, cap 등 있는지 체크)

Check Presence

- 순서 반영이나 변수 타입에 대해 체크하지 않기 때문에 개발 단계에서 간단한 로직 구현 유무로 사용하기에 적합

함수명

해당 함수에서 검사하는 로직

`_has_pause_guard()`

require문에 paused, pause, emergency 키워드가 있는지 체크

`_has_cap_checks()`

supplyCap, borrowCap 검사가 있는지 확인

`_has_interest_call()`

accrueInterest, updateInterest, update가 호출되는지 확인

제안 방법

탐지 기준 설계

Check Presence

- 순서 반영이나 변수 타입에 대해 체크하지 않기 때문에 개발 단계에서 간단한 로직 구현 유무로 사용하기에 적합

Track Order

- 이자 정산이 상태 변경 전에 실행되는지 확인
Checks-Effects-Interactions 패턴 강제

Detect Effects

- transfer, mint, burn, 할당문 등 상태 변경 노드 탐지

함수명

해당 함수에서 검사하는 로직

`_has_pause_guard()`

require문에 paused, pause, emergency 키워드가 있는지 체크

`_has_cap_checks()`

supplyCap, borrowCap 검사가 있는지 확인

`_has_interest_call()`

accrueInterest, updateInterest, update가 호출되는지 확인

`_is_effect_node()`

상태 변경을 일으키는 함수들 탐지 후 실행 흐름이 C → E → I 순서인지 체크

제안 방법

탐지 기준 설계

Precision Loss

- 나누기 먼저, 곱하기 나중 패턴 탐지 (소수점 손실)

Low-Level Calls

- 위험한 .call(), .delegatecall() 사용 탐지

Detect Effects

- transfer, mint, burn, 할당문 등 상태 변경 코드 탐지

함수명

해당 함수에서 검사하는 로직

_has_pause_guard()

require문에 paused, pause, emergency 키워드가 있는지 체크

_has_cap_checks()

supplyCap, borrowCap 검사가 있는지 확인

_has_interest_call()

accrueInterest, updateInterest, update가 호출되는지 확인

_is_effect_node()

상태 변경을 일으키는 함수들 탐지 후 실행 흐름이 C → E → I 순서인지 체크

_has_precision_loss()

accrueInterest, updateInterest, update가 호출되는지 확인

_has_low_level_call()

위험한 .call(), .delegatecall() 사용 탐지

제안 방법

탐지 기준 설계

	AST	AST+CFG	AST+CFG+IR	
Pause 존재	✓	✓	✓	
Cap 검사 존재	✓	✓	✓	
Interest 존재	✓	✓	✓	
실행 순서 검사	✓	✓	✓	
Effect 노드		✓	✓	
CEI 패턴 검사		✓	✓	
Precision Loss			✓	
Low-level Call			✓	

평가 결과

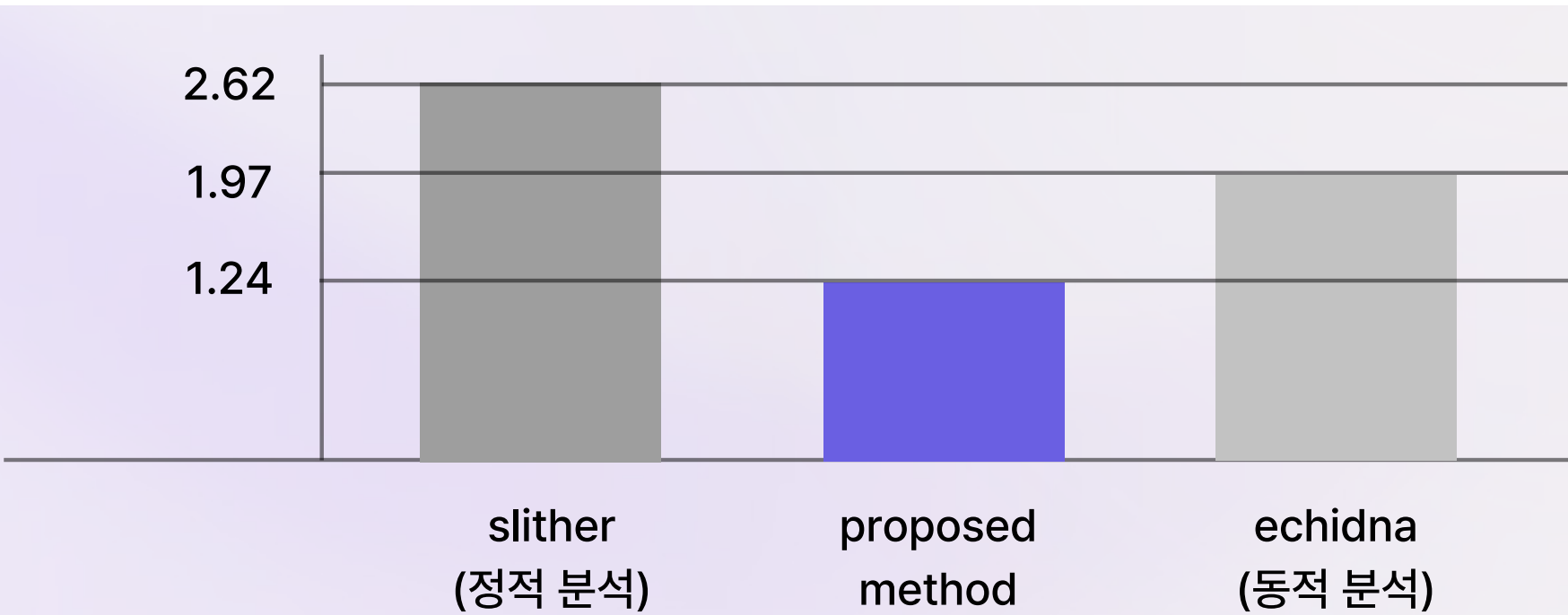
탐지 성능 비교

AST·CFG·IR 기반 다계층 정적 분석 기법이
기존 도구보다 높은 정확도와 빠른 분석 속도를 달성하며,
순서 기반 논리 오류를 안정적으로 탐지함을 확인

계층별 탐지 성능 비교 결과

	Precision	Recall	F1-Score
AST	0.63	1.00	0.78
AST+CFG	0.93	1.00	0.96
AST+CFG+IR	1.00	1.00	1.00

도구 별 평균 분석 시간 비교 결과



결론 및 향후 연구

향후 계획



탐지 범위 확장

다양한 DeFi 프로토콜
(DEX, Staking, Bridge),
최신 컨트랙트로
적용 범위를 확대



자동화된 수정 제안

탐지된 취약점에 대해 가드
로직 삽입 위치와
코드 패치를 자동으로
제안하는 기능을 추가



실시간 모니터링 통합

배포된 컨트랙트의
트랜잭션 패턴을
실시간으로 분석하여
런타임 이상 징후를 탐지